Belk QA - Jenkins Job Configuration

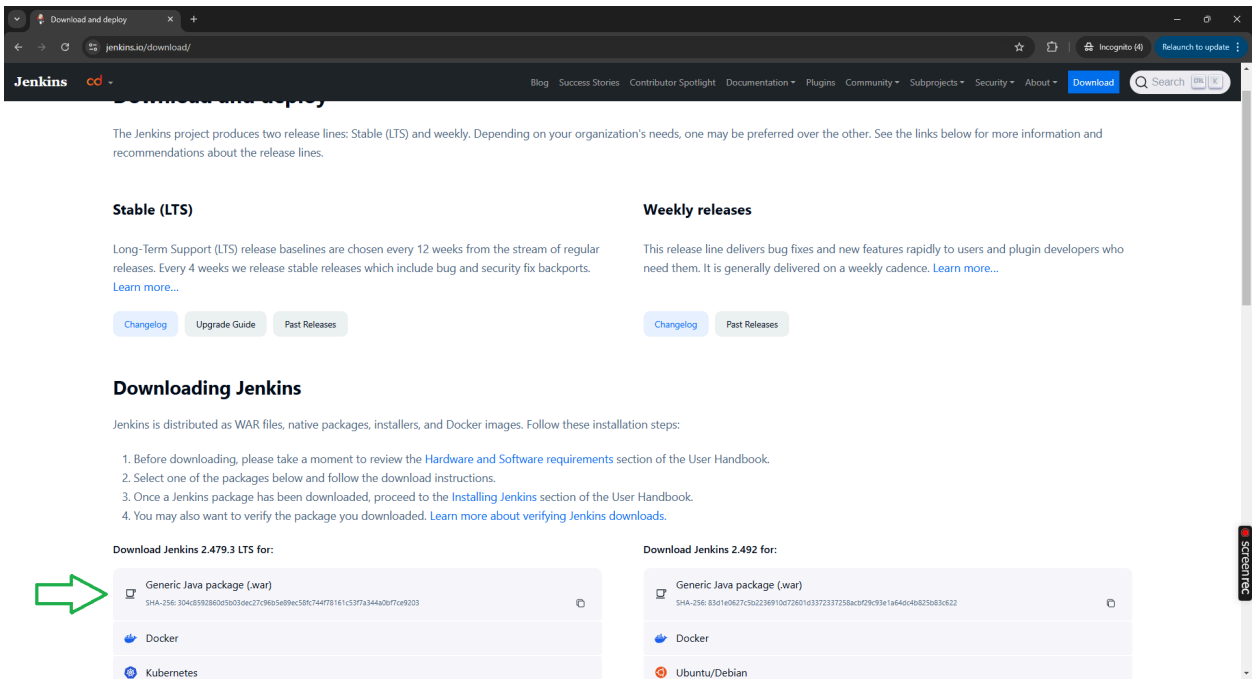
Agenda

1. Download and Set Up Jenkins.....3
2. Configure Global Security and Credentials (LT & Bitbucket).....4
3. Install Required Plugins.....5
4. Configure Managed Files.....5
5. Create a Jenkins Job.....6
6. Create Parameterized Job.....12
7. Execution of Jobs in Jenkins.....15

1. Download and Set Up Jenkins

1. **Download Jenkins WAR File:**

   - Navigate to the official Jenkins website: Jenkins Downloads.
   - Download the jenkins.war file. (By Clicking on Generic Java Package)

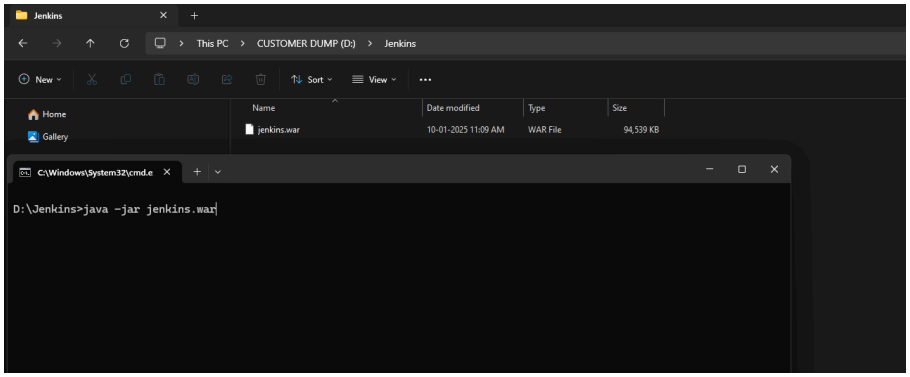


2. **Start Jenkins:**

   - Open a terminal or command prompt.
   - Navigate to the directory containing the jenkins.war file.

   Run the following terminal:

   java -jar jenkins.war



- Jenkins will start and can be accessed at http://localhost:8080.

3. **Unlock Jenkins:**
   - During the first startup, Jenkins will display an unlock key in the terminal.
   - Copy the key and enter it into the Jenkins web interface. (Either from terminal or from the path of C:\Users\<Current-user>\.jenkins\secrets\initialAdminPassword)
4. **Install Suggested Plugins:**
   - Select "Install Suggested Plugins" during the setup wizard.
   - Wait for the plugins to install.
5. **Create Admin User:**
   - Enter the required details to create an admin user.

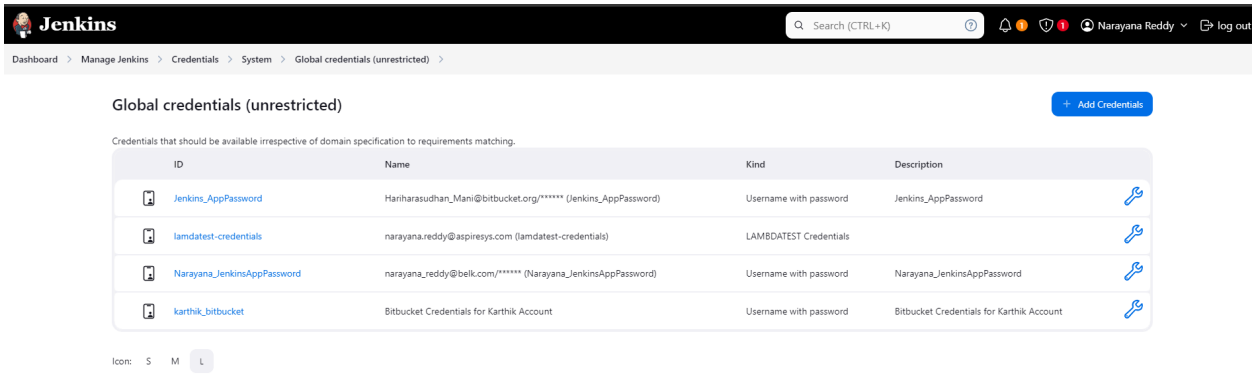
---

2. Configure Global Security and Credentials (LT & Bitbucket)

1. **Global Security Settings: (if required)**

   - Navigate to Manage Jenkins > Configure Global Security.
   - Enable and configure security settings as needed.
2. **Add Credentials:**

   - Navigate to Manage Jenkins > Credentials.
   - Select a domain (e.g., "Global") and add credentials:
     - **Username and Password:** For Bitbucket or other services (Lambdatest)
     - **SSH Key:** For secure access.



---

3. Install Required Plugins

1. Navigate to Manage Jenkins > Plugin Manager.
2. Search for and install the following plugins: (Ignore if the plugins already exist)
    - **Maven Integration Plugin:** For building Maven projects.
    - **Git Plugin:** For integrating with Git repositories.
    - **Pipeline Plugin:** For creating Jenkins pipelines.
    - **Configuration as Code Plugin:** For managing Jenkins configurations programmatically.
    - **Credentials Plugin:** For secure credential management.
    - **Managed Files Plugin:** For handling external configuration files.
    - **LambdaTest Automation Plugin:** For Integrate your Jenkins Jobs to run tests on LambdaTest.
    - **TestNG Results Plugin:** For Store the TestNG Results
    - **JUnit Plugin:** For some graphical visualization of the test results with XML test reports
    - **HTML Publisher Plugin:** For publish HTML reports that your build generates to the job and build pages

---

4. Configure Managed Files

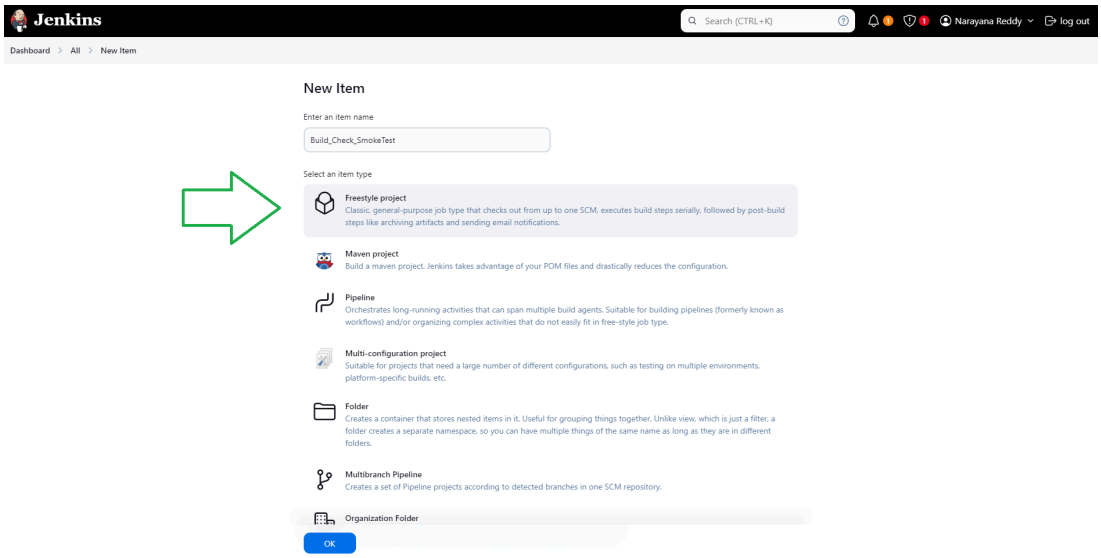
1. Navigate to Manage Jenkins > Managed Files.
2. Add the following files:
    - **Configuration Properties File:**
        - File type: Properties.
        - Upload the required .properties file.
        - Configuration Properties Files Details:
            1. config.properties > Execution Platform/Device Configuration
            2. Cloud_config.properties > LT Credentials Configuration
    - **XML Configuration File:**
        - File type: Custom or XML.
        - Upload the required .xml file.
        - We Can Update the Xml File for below changes,
            1. Required Execution Name
            2. Site URL
            3. Include/Exclude Test Groups
            4. Number of Execution In Parallel

---

5. Create a Jenkins Job

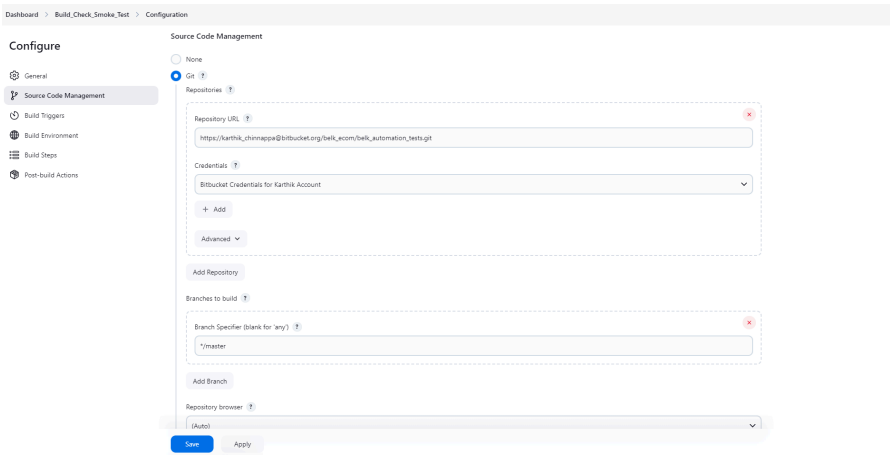
- **New Item:**

  - Navigate to the Jenkins dashboard and click New Item.
  - Enter a name for the job and select Freestyle Project.



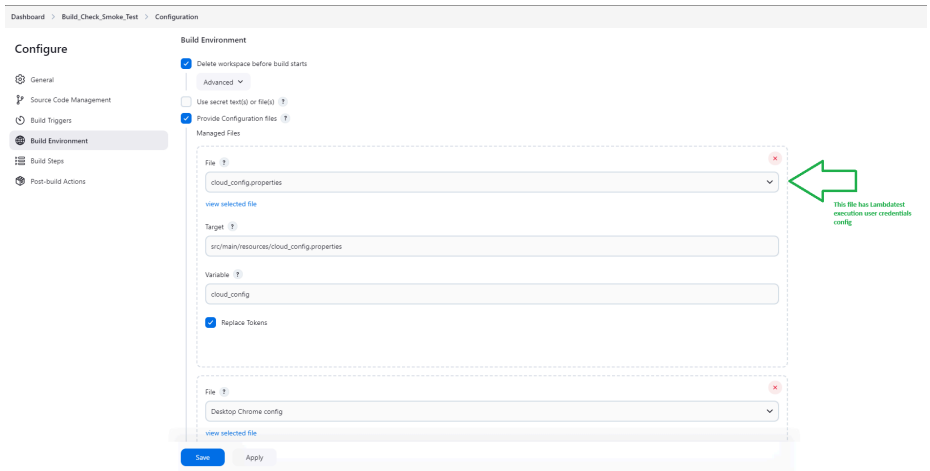
- **Source Code Management (SCM):**

  - Select Git.
  - Add the repository URL from Bitbucket.
  - Use the credentials added earlier.
  - Mention the required branch in Branch Specifier - (e.g., "*/master")

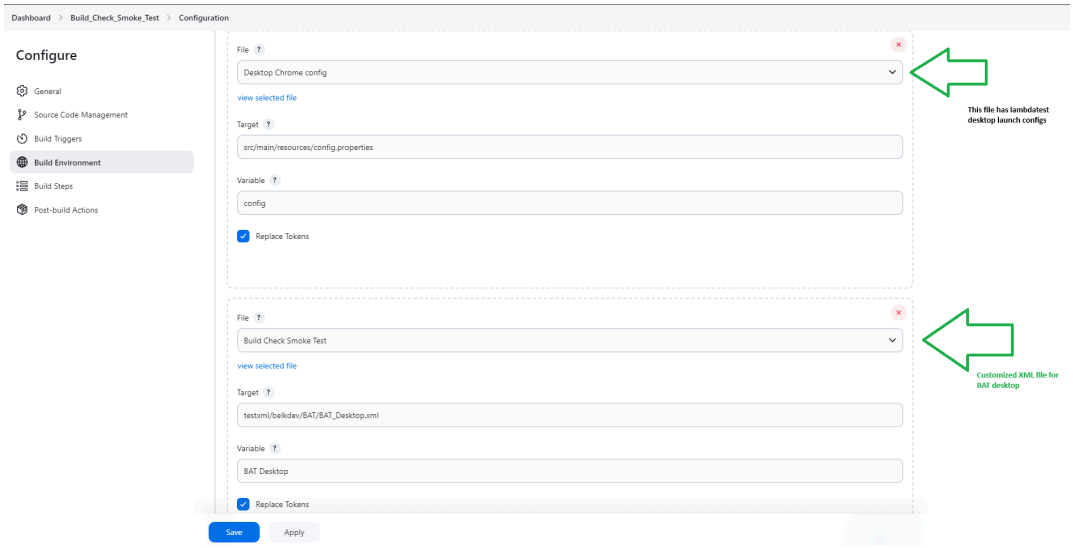


- **Build Environment:**

  - Add the managed configuration properties file under Provide configuration files.
  - Select the Properties files based on requirement of the job that already created in the Jenkins
  - Add the Path of the file in our Code Repository
  - Configuration Files & Path:
    - src/main/resources/config.properties
    - src/main/resources/cloud_config.properties

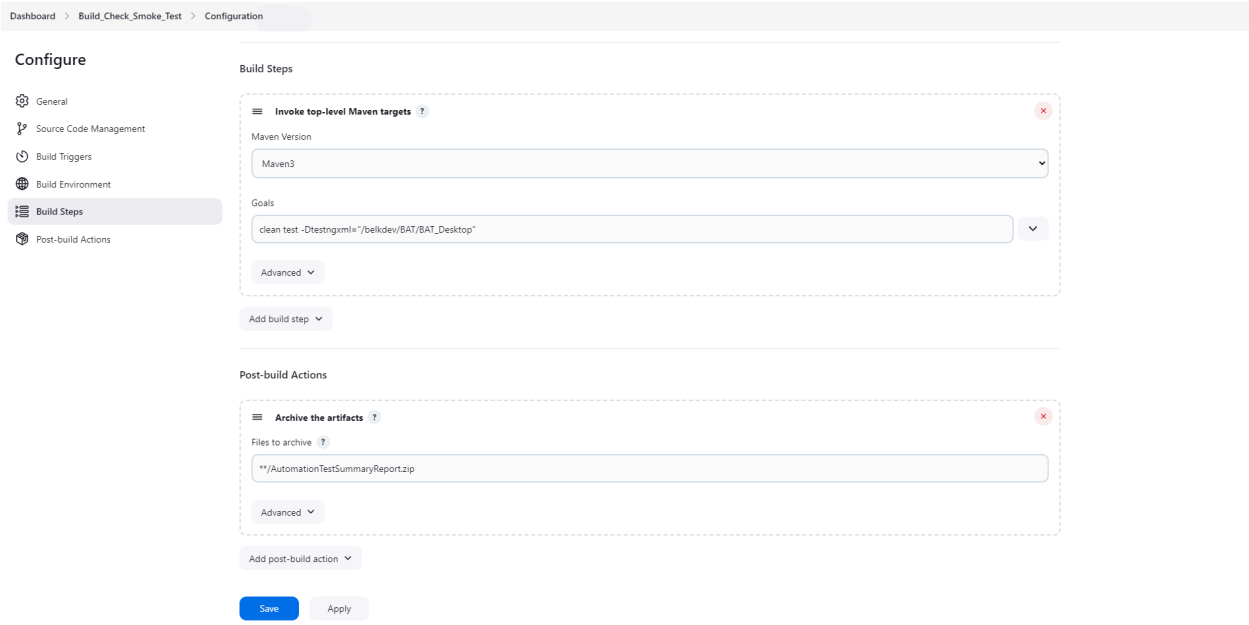


  - Xml Files & Path: (For Example)
    - testxml/belkdev/Search/Search_Desktop.xml
    - testxml/belkdev/Search/Search_Mob.xml
    - testxml/belkdev/Search/Search_Tab.xml

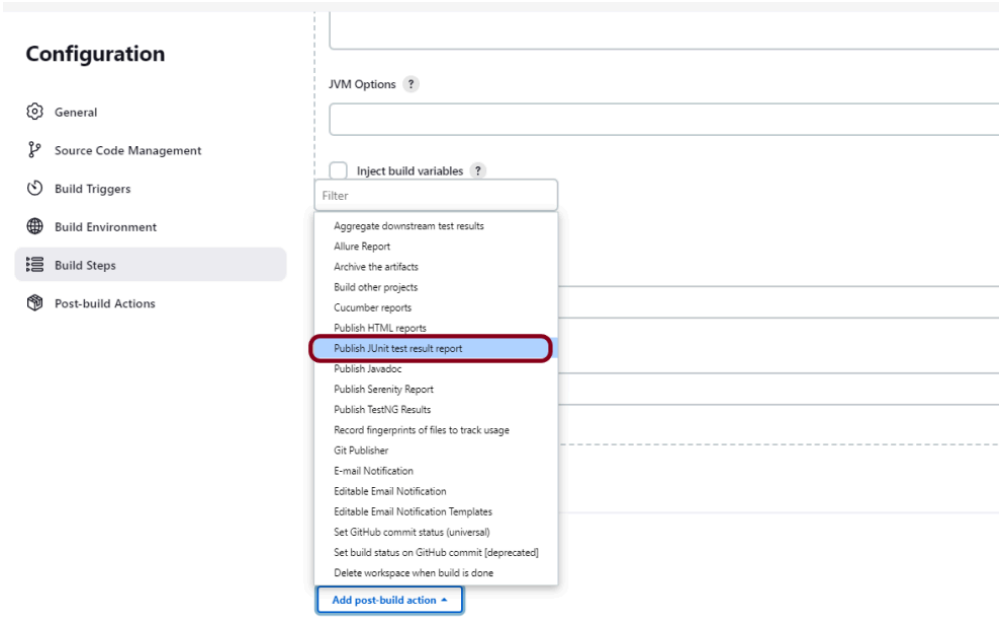


- **Build Steps:**
    - Add a build step to invoke Maven:
        - Enter the Maven goals with required xml File as a Parameter.
        - Maven goals Examples:
            - clean test -Dtestngxml="/belkdev/Search/Search_Desktop"
            - clean test -Dtestngxml="/belkdev/Search/Search_Mob"
            - clean test -Dtestngxml="/belkdev/Search/Search_Tab"
        - Specify the location of the POM file if necessary.

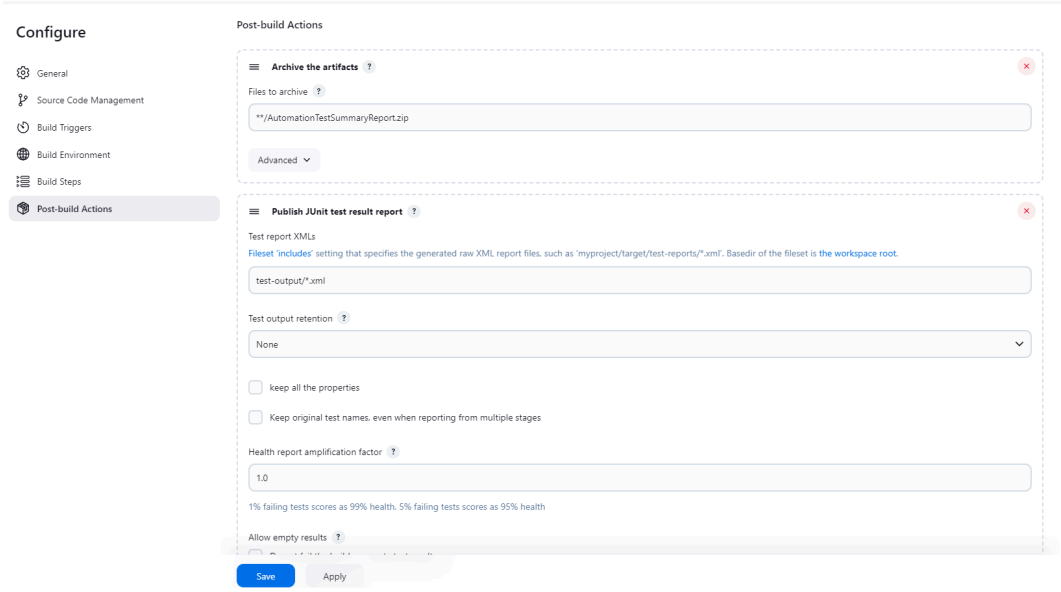
- **Post-build Actions:**
    - Set up archiving for the build artifacts:
        - Navigate to Post-build Actions.
        - Select Archive the artifacts.
        - Specify the folder or file patterns to archive (e.g., target/*.jar).
        - Reports Zip File Path:
            - **/AutomationTestSummaryReport.zip



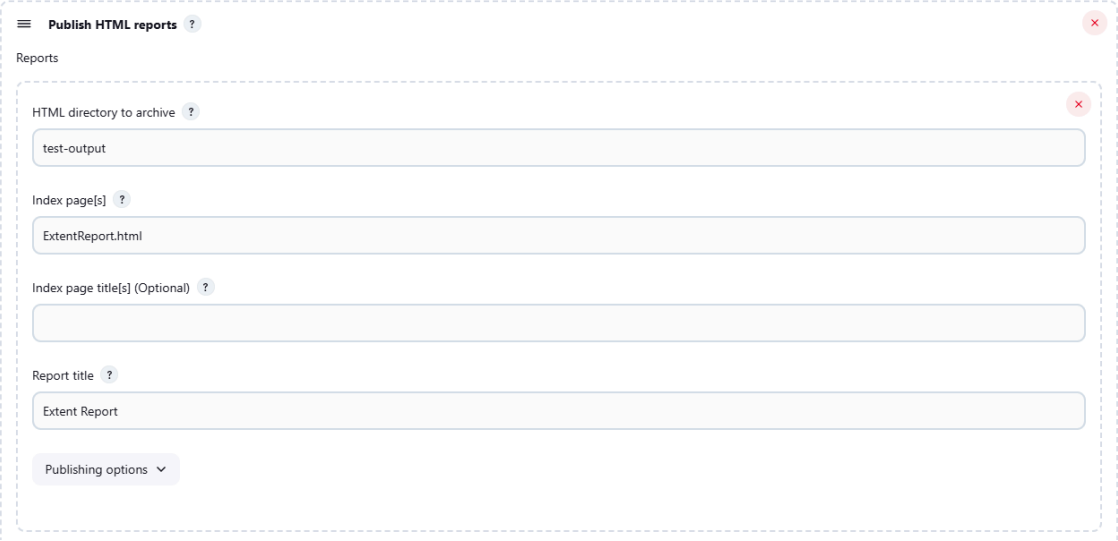
- Select “Publish JUnit test result report” from “Post Build Actions” (Make sure JUnit Results plugin should be installed):
  - Click on the “Add Post Build Actions” drop-down list. Select “Publish JUnit test result report“



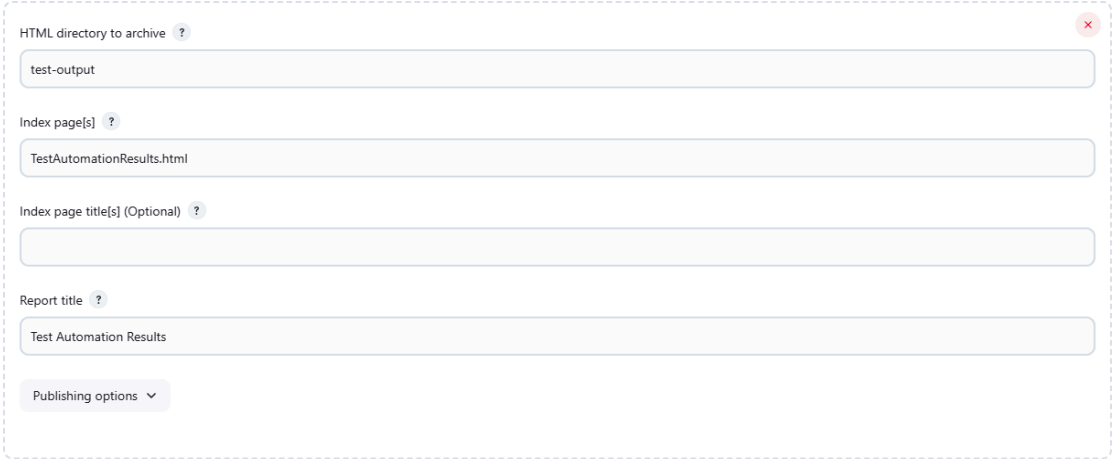
  - Enter the Result Path as “test-output/*.xml”



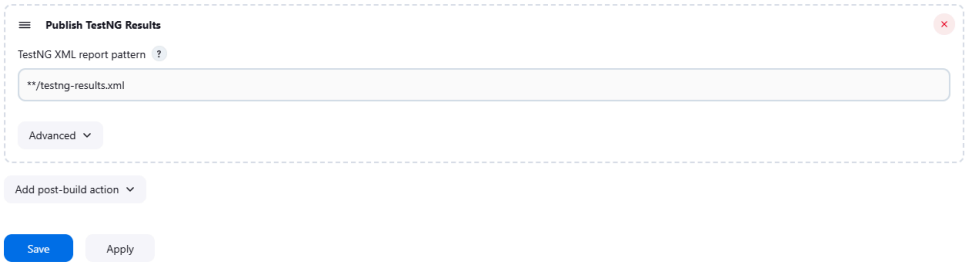
- Select “Publish HTML reports” from “Post Build Actions” (Make sure HTML Publisher plugin should be installed): For “Extend Report”
    - Click on the “Add Post Build Actions” drop-down list. Select “Publish HTML reports“
    - Enter the HTML directory to archive as “test-output”
    - Enter the Index page[s] as “ExtentReport.html”
    - Enter the Report Title as “Extent Report”



- Select “Publish HTML reports” from “Post Build Actions” (Make sure HTML Publisher plugin should be installed): For “Test Automation Results”
    - Click on the “Add Post Build Actions” drop-down list. Select “Publish HTML reports“
    - Enter the HTML directory to archive as “test-output”
    - Enter the Index page[s] as “TestAutomationResults.html”
    - Enter the Report Title as “Test Automation Results”



- Select “Publish TestNG Results” from “Post Build Actions” (Make sure TestNG Results plugin should be installed):
  - Click on the “Add Post Build Actions” drop-down list. Select “Publish TestNG Results“
  - Enter the TestNG XML report pattern as “**/testng-results.xml”



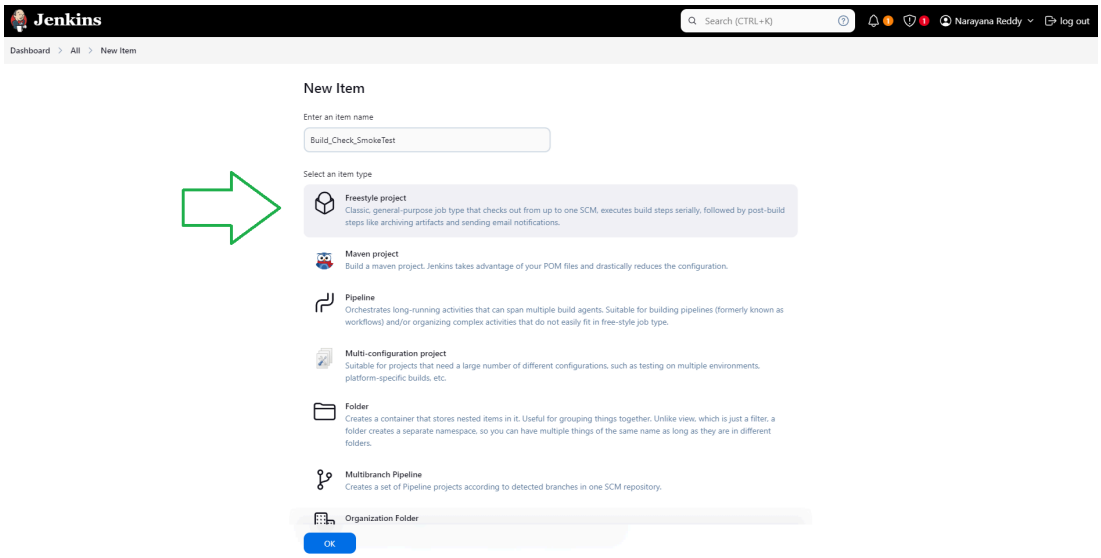
  - Click on the Apply and Save buttons

---

6. Create Parameterized Job

- **New Item:**

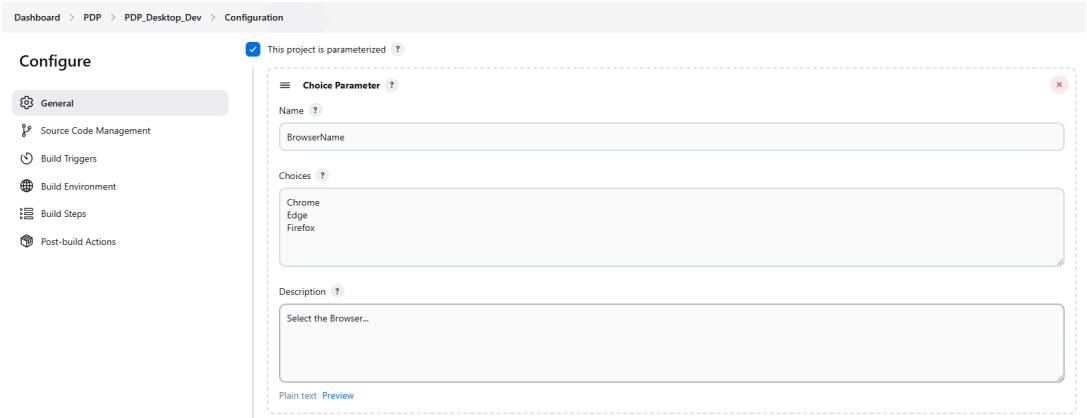
  - Navigate to the Jenkins dashboard and click New Item.
  - Enter a name for the job and select Freestyle Project.



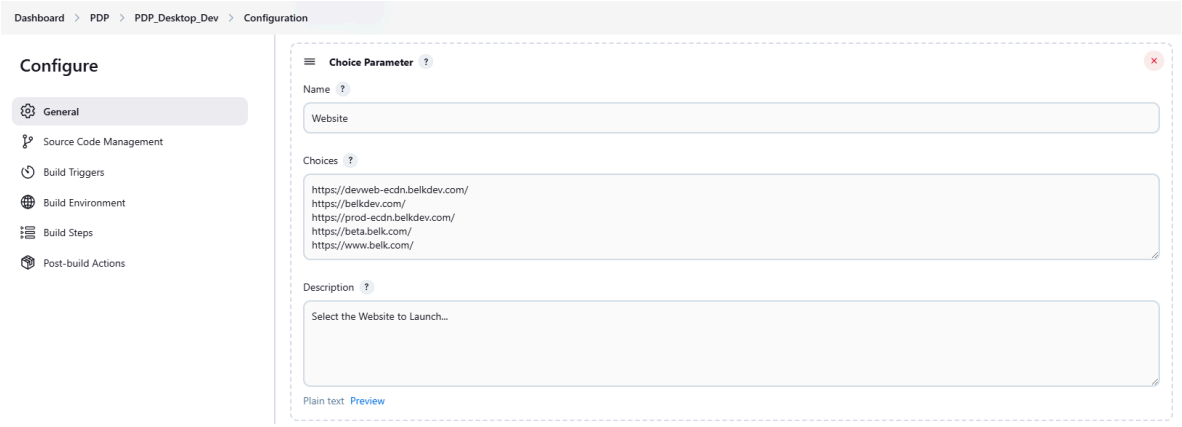
- **Source Code Management (SCM):**

  - Select Git.
  - Add the repository URL from Bitbucket.
  - Use the credentials added earlier.
  - Mention the required branch in Branch Specifier - (e.g., "*/master")

- **Add Browser Parameter:**
  - Select “This project is parameterize”
  - Click “Add Parameter” dropdown
  - Select “Choice Parameter” option
  - Enter the option Name as “BrowserName”
  - Under the Choice textbox enter “Chrome Edge Firefox”

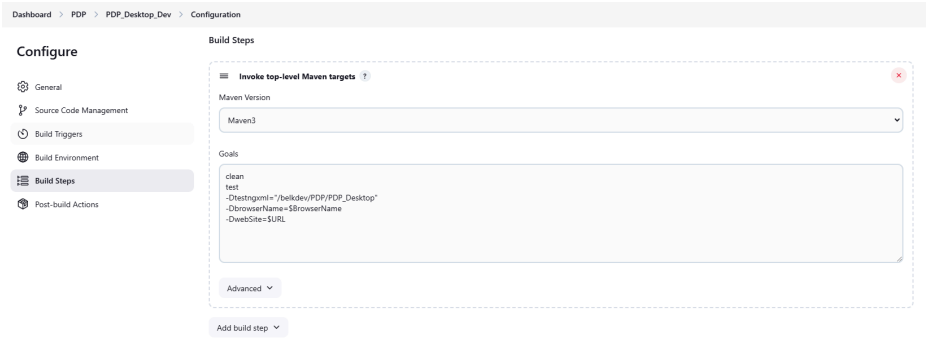


- **Add Site URL Parameter:**
  - Select "This project is parameterize"
  - Click "Add Parameter" dropdown
  - Select "Choice Parameter" option
  - Enter the option Name as "Website"
  - Under the Choice textbox enter
    - "https://devweb-ecdn.belkdev.com/"
    - "https://belkdev.com/"
    - "https://prod-ecdn.belkdev.com/"
    - "https://www.belk.com/"

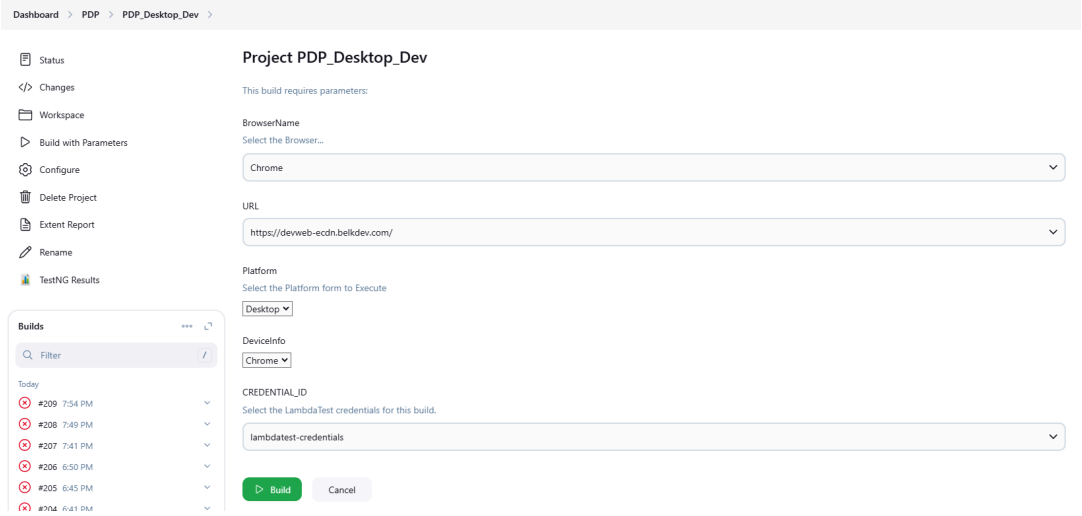


- **Config Build Steps with Parameters**
  - Select Maven Version “Maven3”
  - Enter Goals as

    “clean test -Dtestngxml="/belkdev/PDP/PDP_Desktop"
    -DbrowserName=$BrowserName
    -DwebSite=$URL”



- **Execute the Build with Parameter**
  - Click the “Build with Parameter” Option in the Job
  - Select the “Browser Name” & “Site URL” from the dropdown options
  - Click “Build” button

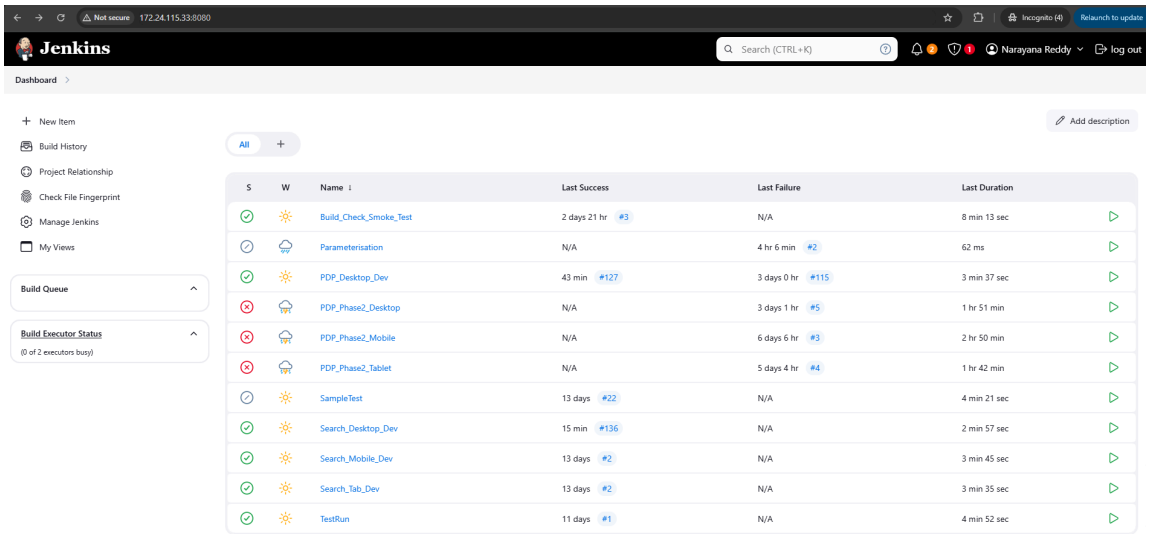


7. Execution of Jobs in Jenkins

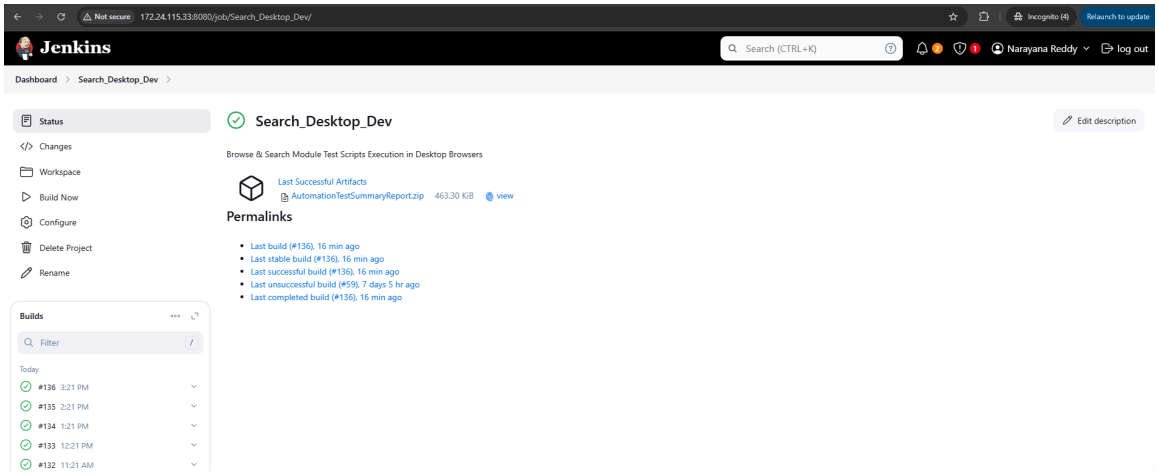
1. Start the jenkins.war file through Command prompt by using “**java -jar jenkins.war**".

```
E:\Software>java -jar jenkins.war
Running from: E:\Software\jenkins.war
```

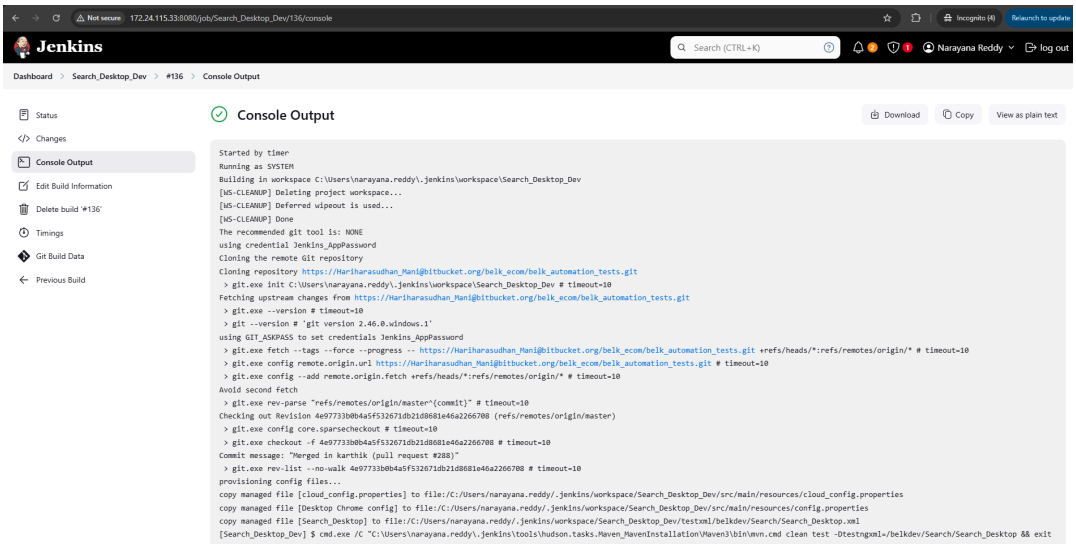
2. Connect to Jenkins by going to the following URL http://<hostname>:8080/ or by default URL is http://localhost:8080/ (if have username and Password login)
3. Click the Required (e.g., Search_Desktop_Dev) Job file.



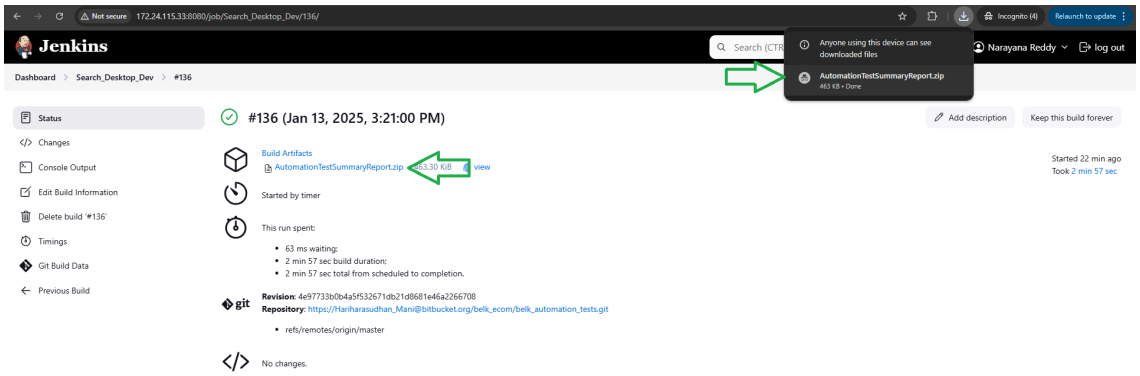
4. Click the “Build Now” option in the Left side Panel, then it will start execution.

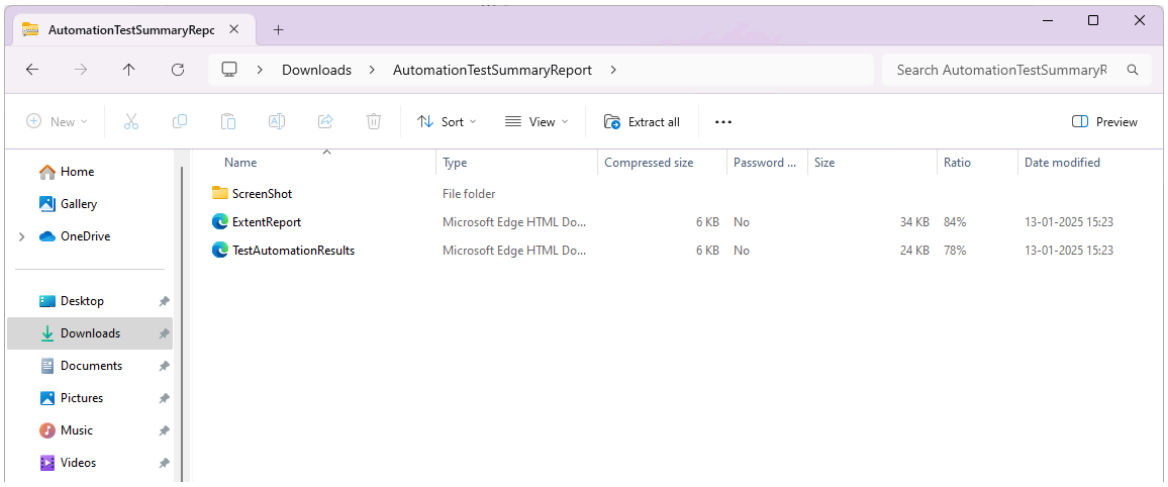


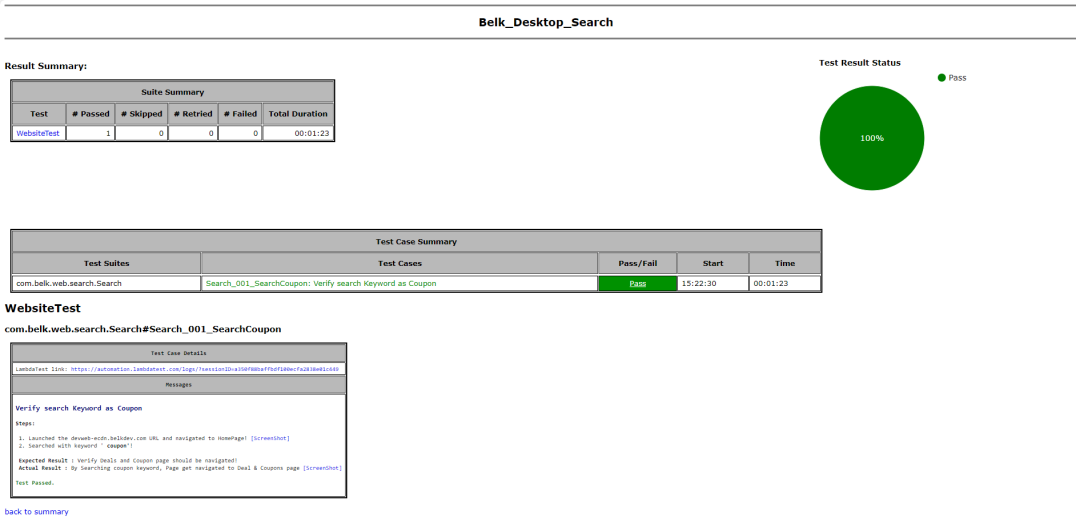
5. It is hitting the triggered build then go through the Console Output option and see the execution.

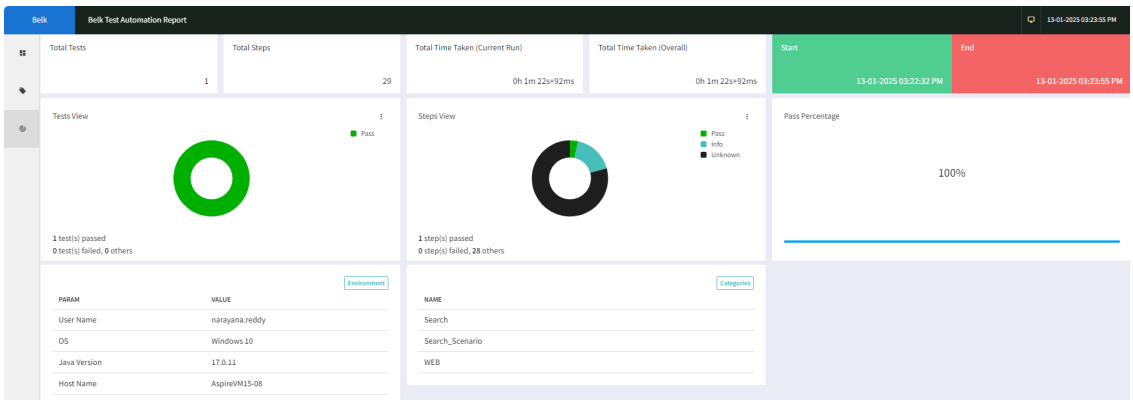


6. Finally build is given to determine if the result is successful or failure will check the reports on Html Reports. see the below screens

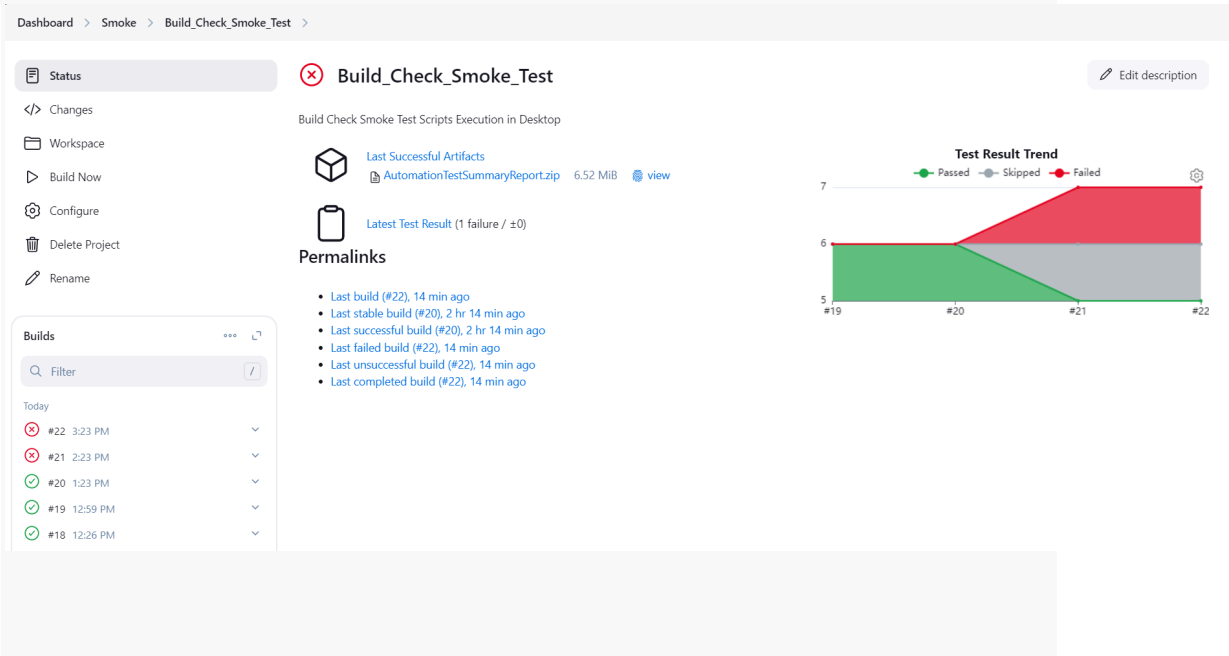




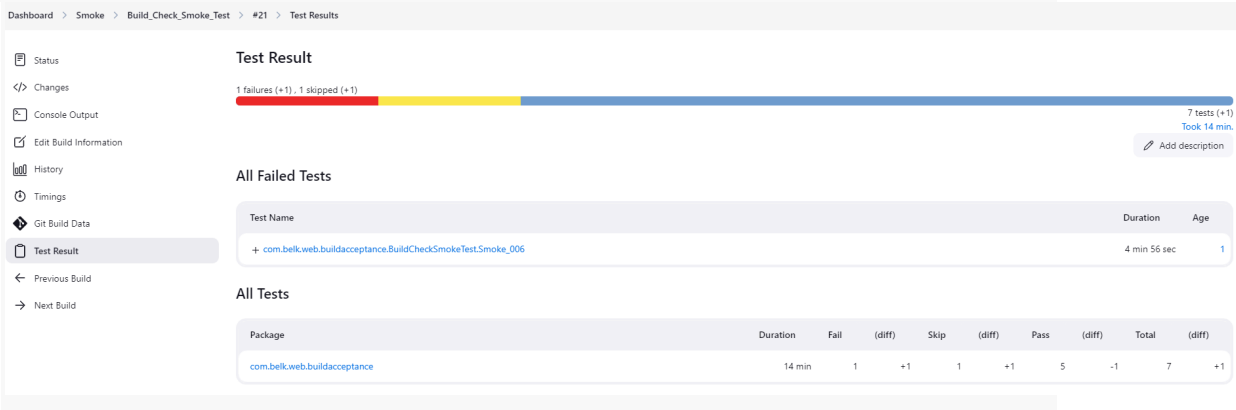




7. View the JUnit Report
   a. Once the execution is completed, we could see a link to view the “Test Report“

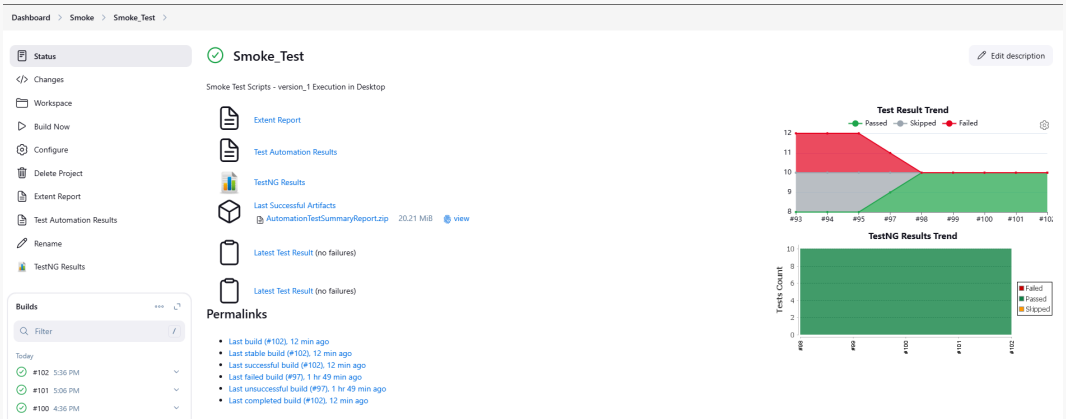


b. Below is the summary of the Specific Build's Test Execution Report

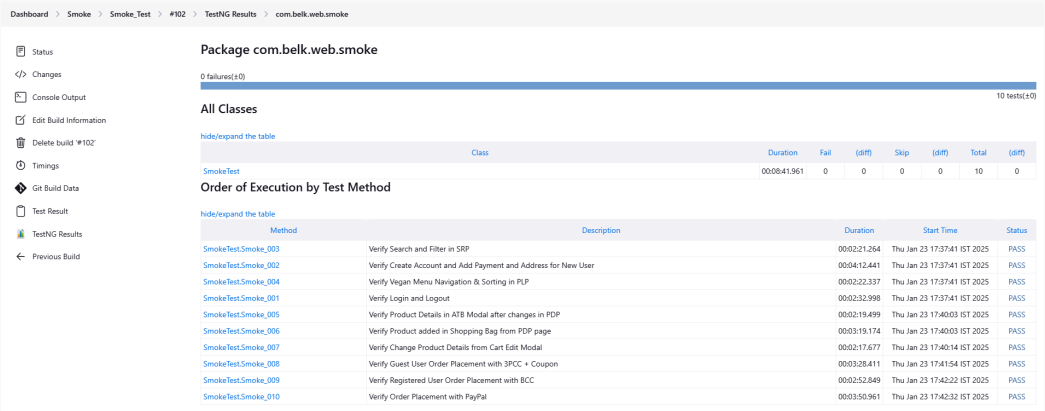


8. View TestNG Reports

a. Once the execution is completed, we could see a link to view the "TestNG Results"

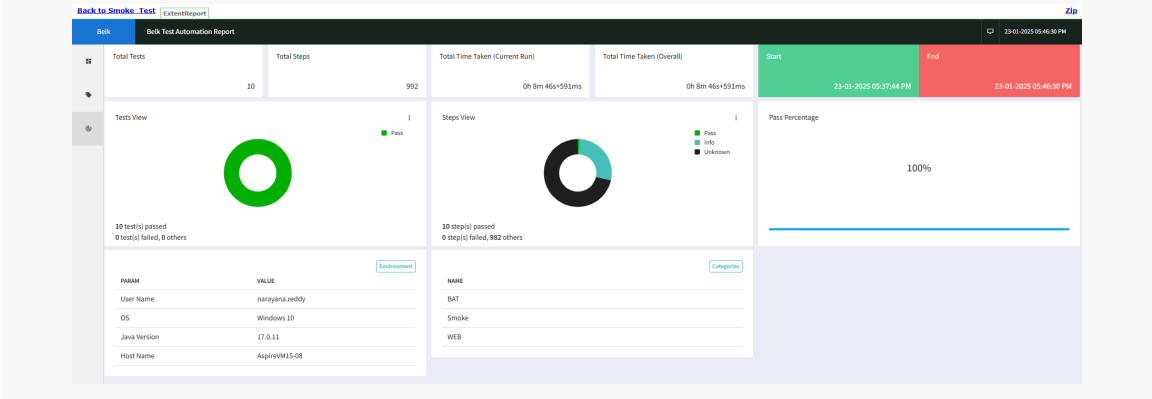


b. Below is the summary of the Specific Build's Test Execution Report

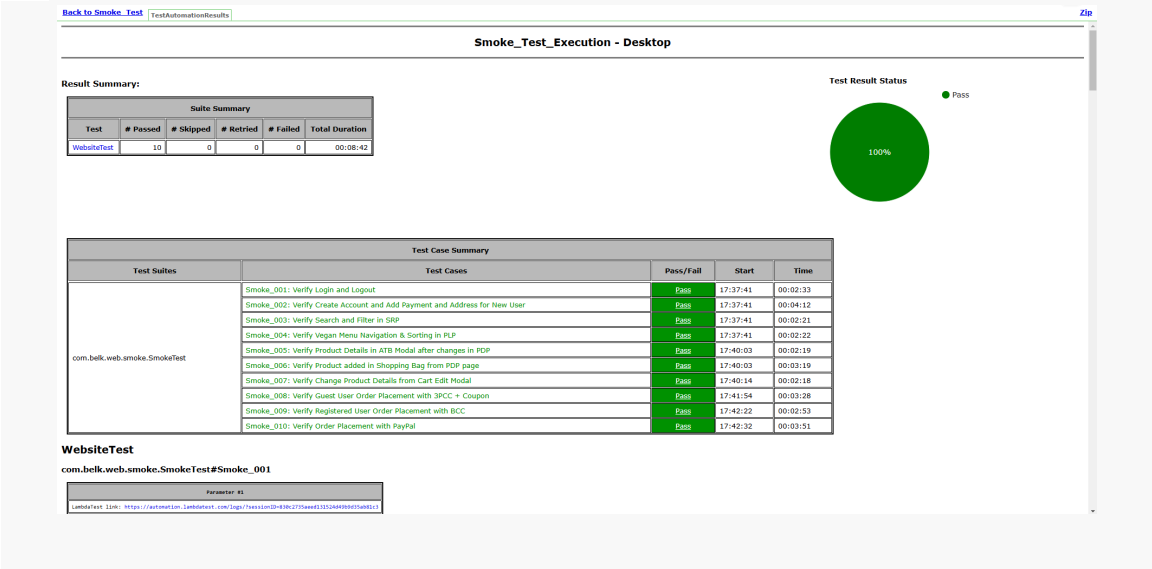


9. View HTML Reports

a. Once the execution is completed, we could see a link to view the "Extend Report"



b. “Test Automation Results” HTML Report also will be available in Job Dashboard



10. Run the job and ensure:
    a. The repository is correctly cloned.
    b. The managed files are available in the workspace.
    c. The Maven build runs successfully.
    d. The artifacts are archived as expected.